Primer proyecto: Monoambiente

Iker Sebastian Ibarra - Giuliano Rosso

Computación Gráfica - 1er cuatrimestre 2026

Objetivo principal

Construir en Unity un monoambiente con una escala realista, cargando los objetos 3D mediante un parser propio de archivos .obj, y renderizando mediante el uso de las matrices de transformación vistas en clase:

- Matriz de modelado.
- Matriz de vista.
- Matriz de proyeccion.

Implementar dos tipos de cámaras:

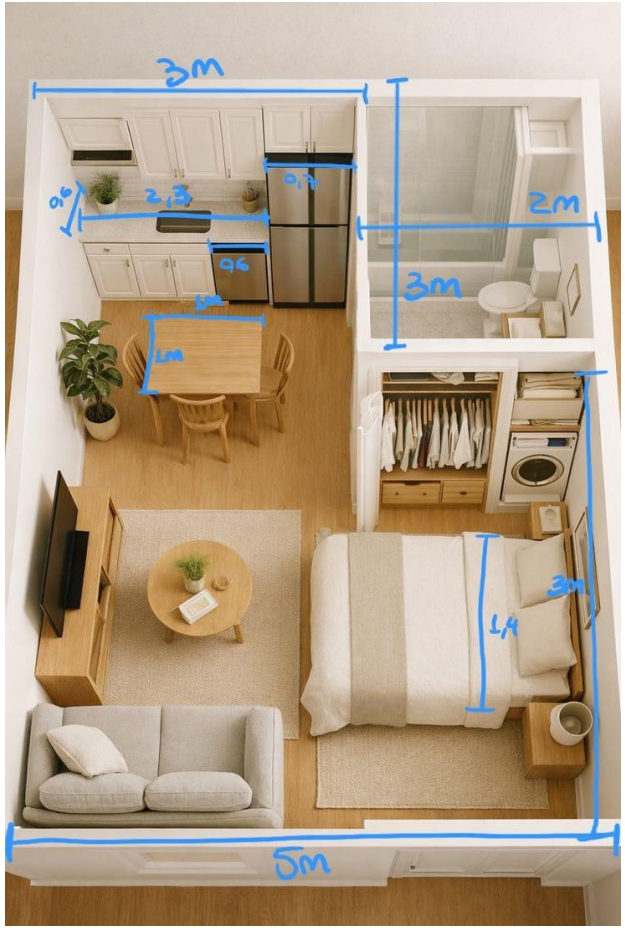
- Cámara en primera persona.
- Cámara orbital.

Nuestra idea inicial:

Con una medida de: 5x6 m, 30 m²

Nuestra primeras pruebas con paredes y suelo hardcodeados

Luego realizamos la migración a Blender + exportación a .obj

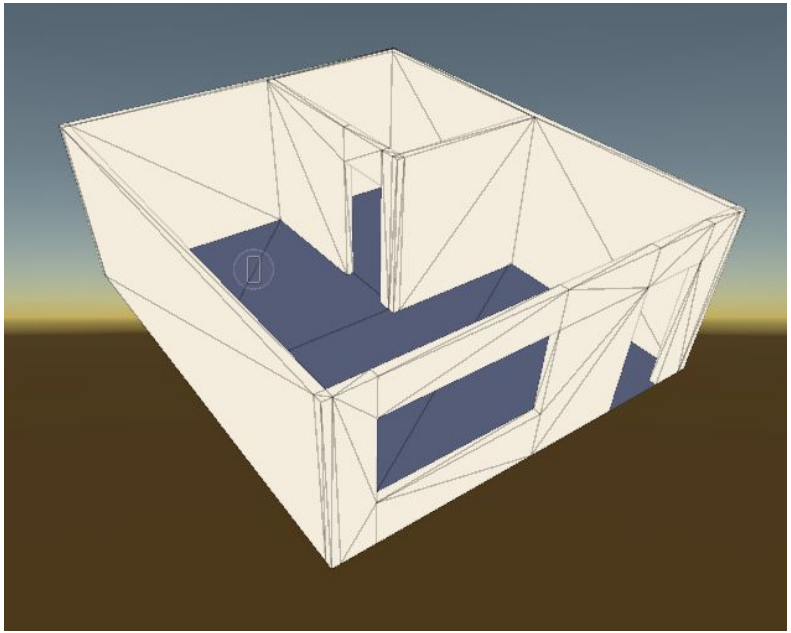


Nuestra idea inicial:

Con una medida de: 5x6 m, 30 m²

Nuestra primeras pruebas con paredes y suelo hardcodeados

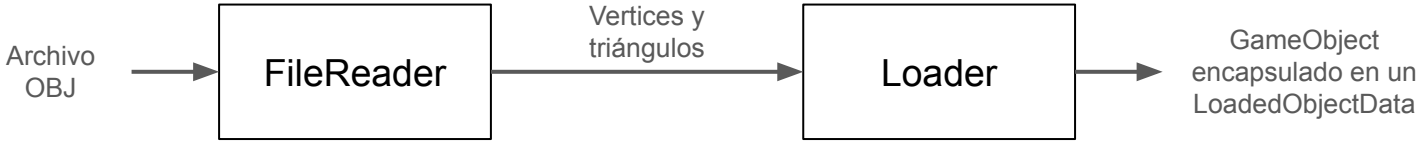
Luego realizamos la migración a Blender + exportación a .obj



El parser de OBJ

Acá nos convenia separar el problema en 2 partes:

- Leer vértices y caras del .obj
- Transformar esa info en una mesh de Unity

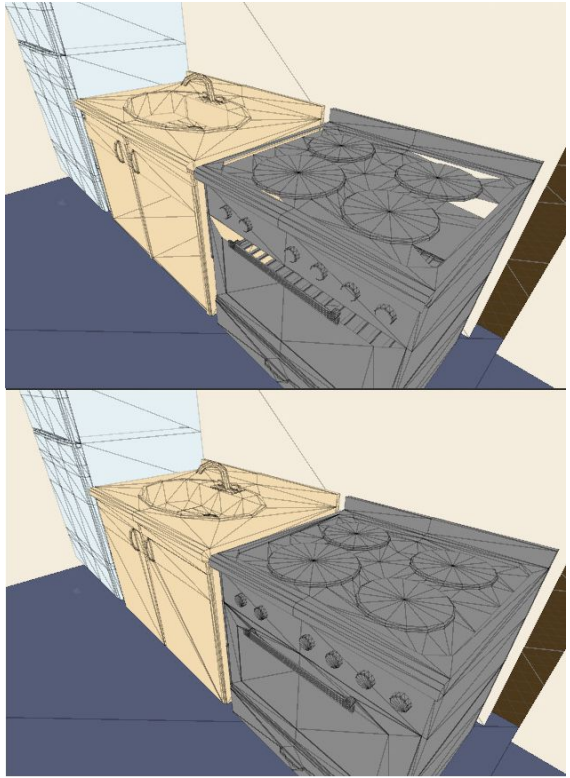


El parser de OBJ

Algunos de los problemas que tuvimos:

Al principio el parser soportaba una cantidad fija de vértices (3 para triangulos o 4 para planos).

➔ Lo rediseñamos para triangular caras de n vértices



El parser de OBJ

Algunos de los problemas que tuvimos:

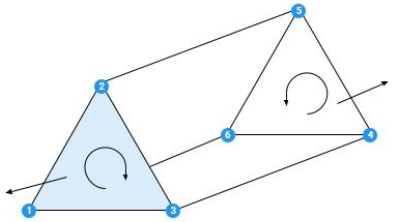
Unity interpretaba el orden de caras distinto al del .obj

➔ Tuvimos que invertir el recorrido de índices

Unity Documentation Manual Scripting API

Winding order

The order of the vertices in each group in the index array is called the **winding order**. Unity uses winding order to determine whether a face is front-facing or back-facing, and in turn whether it should render a face or cull it (exclude it from rendering). By default, Unity renders front-facing polygons and culls back-facing polygons. Unity uses a clockwise winding order, which means that Unity considers any face where the indices connect in a clockwise direction is front facing.



Unity's winding order visualized on a prism

Wavefront .obj file

Article Talk Tools

From Wikipedia, the free encyclopedia

OBJ (or .OBJ) is a geometry definition file format first developed by Wavefront Technologies for *The Advanced Visualizer* animation package. It is an open file format and has been adopted by other 3D computer graphics application vendors.

The OBJ file format is a simple data-format that represents 3D geometry alone – namely, the position of each vertex, the UV position of each texture coordinate vertex, vertex normals, and the faces that make each polygon defined as a list of vertices, and texture vertices. Vertices are stored in a counter-clockwise order by default, making explicit declaration of face normals unnecessary. OBJ coordinates have no units, but OBJ files can contain scale information in a human readable comment line.

El parser de OBJ

Algunos de los problemas que tuvimos:

Unity interpretaba el orden de caras distinto al del .obj

➔ Tuvimos que invertir el recorrido de índices

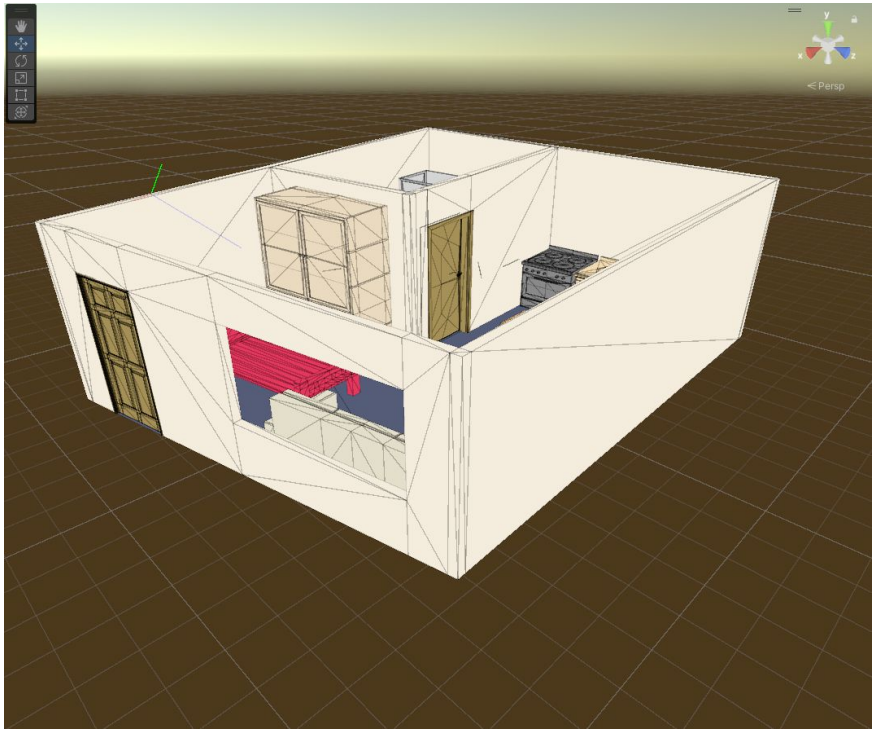
Unity renders front-facing polygons and culls back-facing polygons. Unity uses a clockwise winding order, which means that Unity considers any face where the indices connect in a clockwise direction is front facing.

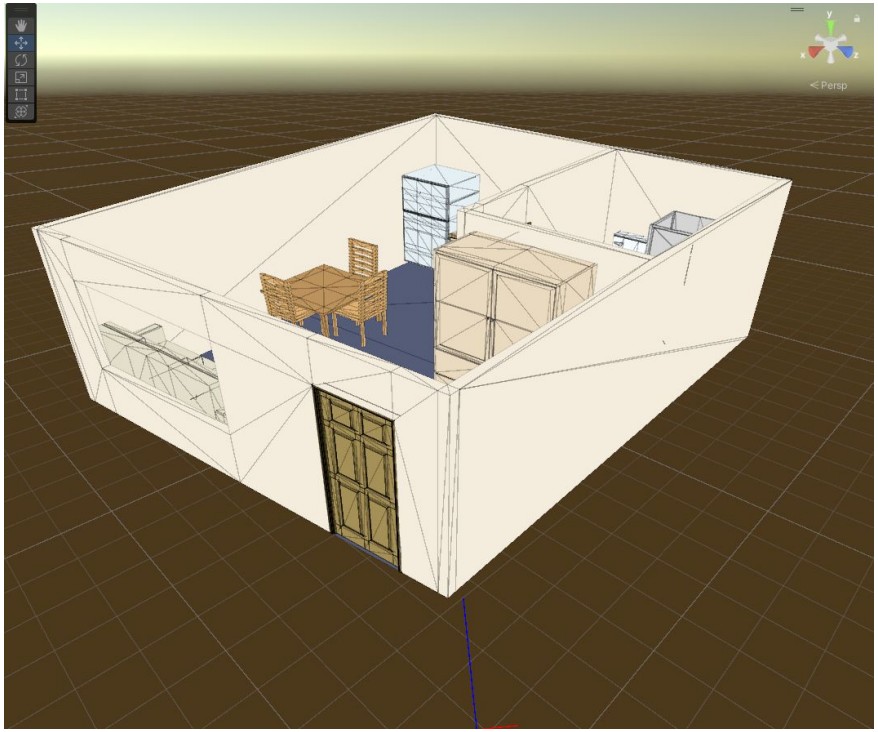
Según Unity

The OBJ file format is a simple data-format that represents 3D geometry alone – namely, the position of each vertex, the UV position of each texture coordinate vertex, vertex normals, and the faces that make each polygon defined as a list of vertices, and texture vertices. Vertices are stored in a counter-clockwise order by default, making explicit declaration of face normals unnecessary. OBJ coordinates have no units, but OBJ files can contain scale

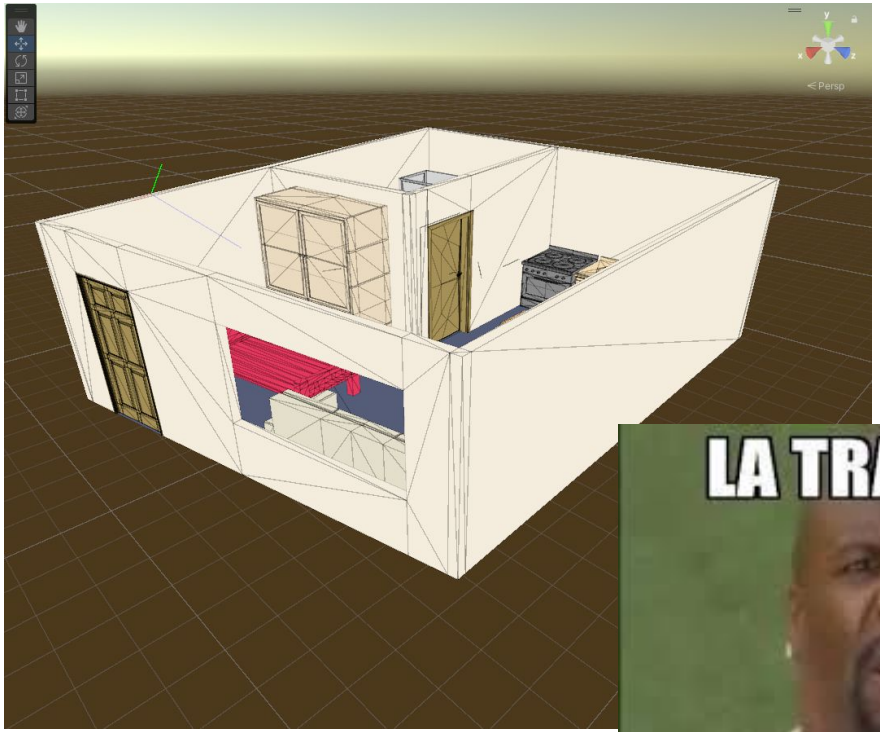
Según el OBJ

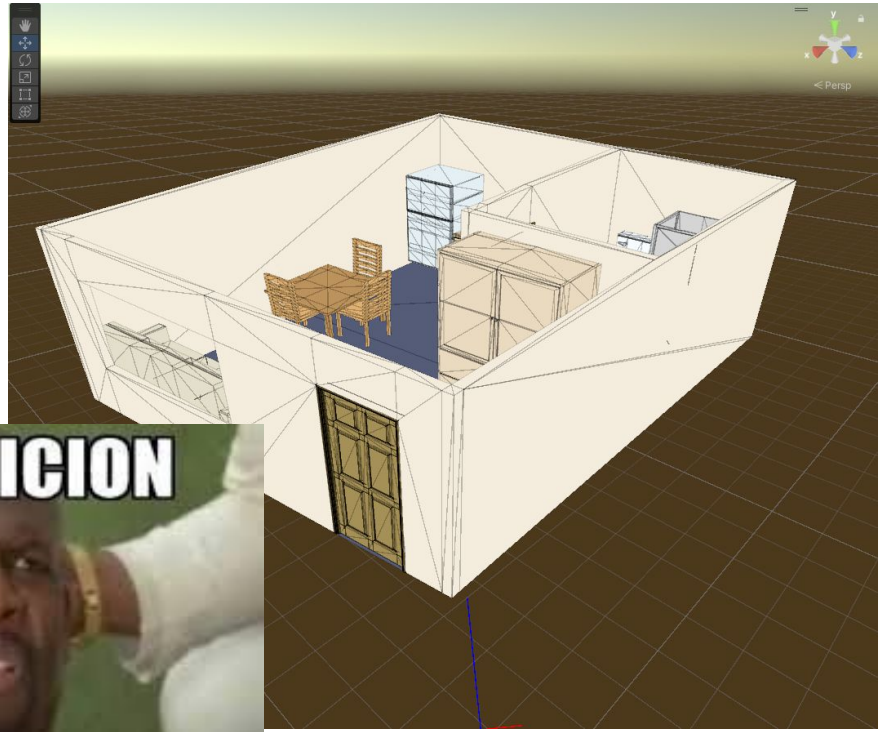
El parser de OBJ





El parser de OBJ

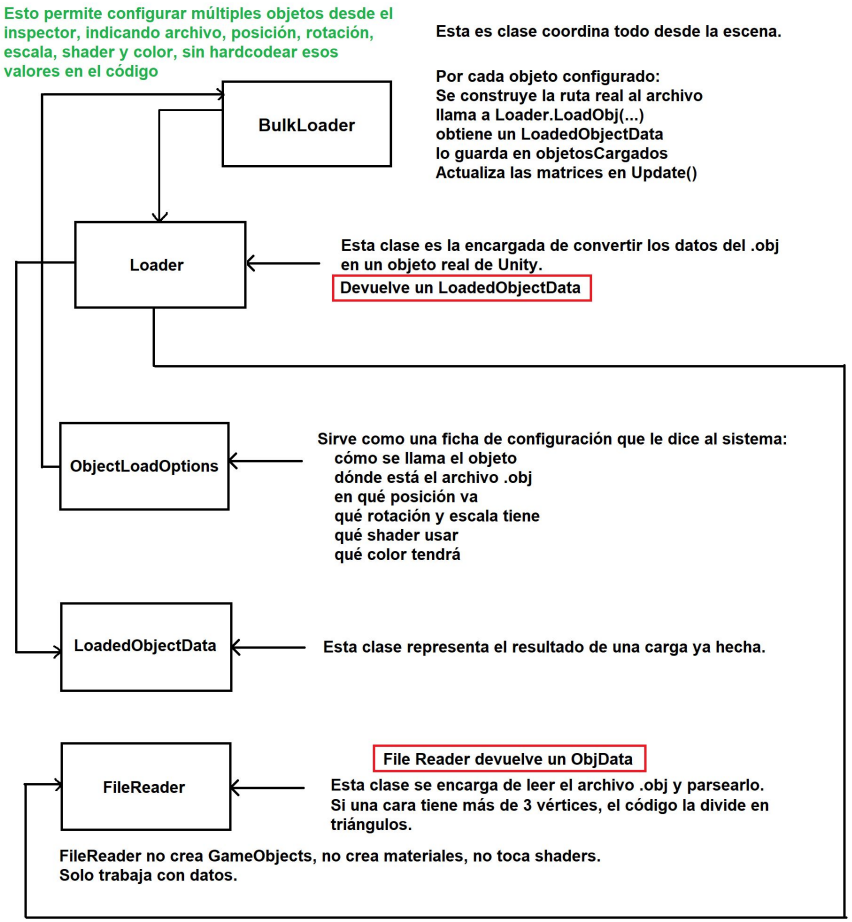




Cargar y posicionar objetos en escena

Acá entra el Bulk Loader.

Ejemplificamos el funcionamiento con el siguiente diagrama:



¿Como utilizamos las matrices vistas en clase?

Primero, carga de objetos desde .obj

Posicionamiento en la escena

Uso de matriz de vista y proyección, que es actualizada en el Update() de BulkLoader

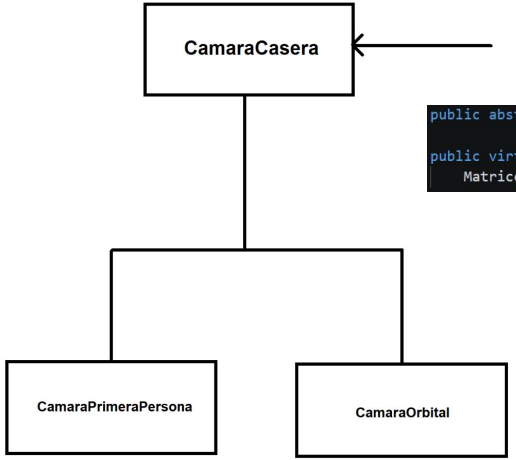
Actualización de matrices por objeto

Actualizamos la matriz de modelado en el Update para posicionar los objetos mientras corremos el programa

```
void Update()
{
    var curCam = GameManager.instance.currentCamera;
    var vm = curCam.GetViewMatrix();
    var pm = curCam.GetProjectionMatrix();

    foreach (var obj in objetosCargados)
    {
        var material = obj.MeshRenderer.material;
        var orig = objetosACargar.Find(o => o.name == obj.Name);
        var pos = orig.position;
        var rot = orig.rotation;
        var sca = orig.scale;

        var mm = Matrices.CreateModelMatrix(pos, rot, sca);

        // Por ahora las posiciones de los objetos no cambian,
        // por lo que no es necesario actualizar su ModelMatrix
        material.SetMatrix("_ModelMatrix", mm);
        material.SetMatrix("_ViewMatrix", vm);
        material.SetMatrix("_ProjectionMatrix", pm);
    }
}
```

Sistema de camaras

No solo hicimos que la cámara funcione, sino que armamos una arquitectura reutilizable para distintos tipos de vista.

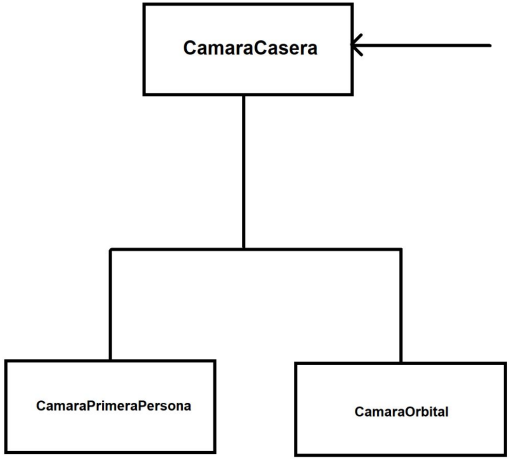


Clase Abstracta con parametros para la camara en primera persona y orbital

Metodos Abstractos:

```
public abstract Matrix4x4 GetViewMatrix();

public virtual Matrix4x4 GetProjectionMatrix() =>
    Matrices.CreateProjectionMatrix(fov, aspectRatio, nearClip, farClip);
```

Sistema de camaras



Clase Abstracta con parametros para la camara en primera persona y orbital

```
public abstract Matrix4x4 GetViewMatrix();

public virtual Matrix4x4 GetProjectionMatrix() =>
    Matrices.CreateProjectionMatrix(fov, aspectRatio, nearClip, farClip);
```

```
// CameraOrbital.cs
public override Matrix4x4 GetViewMatrix()
{
    return Matrices.CreateViewMatrixFromTargetPoint(position, target, Vector3.up);
}
```

```
// CameraFP.cs
public override Matrix4x4 GetViewMatrix()
{
    return Matrices.CreateViewMatrixFromSphericalCoords(position, yawPitch);
}
```

Shaders

Usamos el shader básico visto en clase.

```
CGPROGRAM

#pragma vertex vert
#pragma fragment frag

struct appdata
{
    float4 vertex : POSITION;
    fixed4 color: COLOR;
};

struct v2f
{
    float4 vertex : SV_POSITION;
    fixed4 color: COLOR;
};

uniform float4x4 _ModelMatrix;
uniform float4x4 _ViewMatrix;
uniform float4x4 _ProjectionMatrix;

v2f vert(appdata v)
{
    v2f o;
    o.vertex = mul(mul(_ProjectionMatrix, mul(_ViewMatrix, _ModelMatrix)), v.vertex);
    o.color = v.color;

    return o;
}

half4 frag(v2f i) : SV_Target
{
    return(i.color);
}
ENDCG
```

Shaders

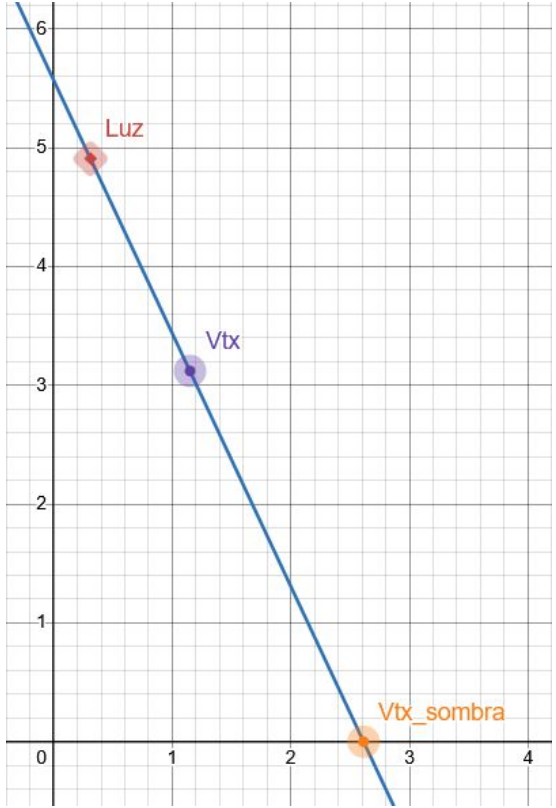
Usamos el shader básico visto en clase.

Pero queríamos probar a hacer algo más complicado.

```
CGPROGRAM

#pragma vertex vert
#pragma fragment frag

struct appdata
{
    float4 vertex : POSITION;
    fixed4 color: COLOR;
};

struct v2f
{
    float4 vertex : SV_POSITION;
    fixed4 color: COLOR;
};

uniform float4x4 _ModelMatrix;
uniform float4x4 _ViewMatrix;
uniform float4x4 _ProjectionMatrix;

v2f vert(appdata v)
{
    v2f o;
    o.vertex = mul(mul(_ProjectionMatrix, mul(_ViewMatrix, _ModelMatrix)), v.vertex);
    o.color = v.color;

    return o;
}

half4 frag(v2f i) : SV_Target
{
    return(i.color);
}
ENDCG
```

Shaders

Queríamos ampliar el shader básico que vimos para implementar *sombras.*



Shaders

Queríamos ampliar el shader básico que vimos para implementar *sombras.*

```
v2f vert(appdata vt)
{
    v2f o;

    float4 worldPos = mul(_ModelMatrix, vt.vertex);
    float3 lightWorldPos = float3(0.5, 2.4, 0.7);

    float a = lightWorldPos.x;
    float b = lightWorldPos.y;
    float c = lightWorldPos.z;
    float u = worldPos.x;
    float v = worldPos.y;
    float w = worldPos.z;

    worldPos.y = 0.005;
    worldPos.x = (a*v - b*u) / (v - b);
    worldPos.z = (c*v - b*w) / (v - b);

    o.vertex = mul(mul(_ProjectionMatrix, _ViewMatrix), worldPos);
    o.color = vt.color;

    return o;
}
```

Resultado

Movimiento

Con el **1** se entra al modo Orbital:

- Se arrastra la vista haciendo **click derecho + moviendo el mouse**.
- Se hace zoom con la **rueda del mouse**.

Con el **2** se entra al modo Primera Persona:

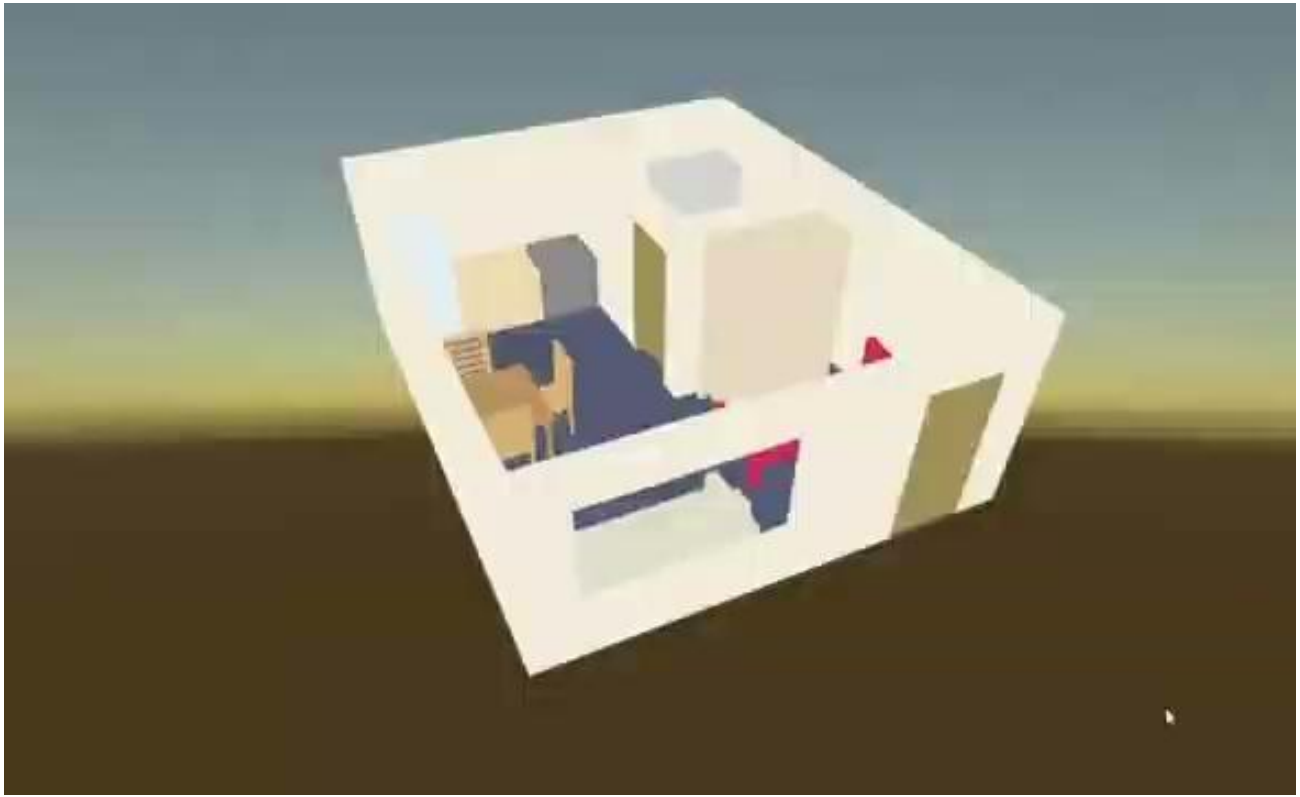
- Se mueve la dirección de la vista con el **mouse**.
- Se cambia la posición con **WASD** o con las **flechas**.
- Se hace zoom con el **Alt**.
- Se mueve mas rápido con **Shift** y mas lento con **Ctrl**.

Con el **9** se activa/desactiva el techo.

Con el **8** se activan/desactivan las paredes.

Con el **7** se activan/desactivan las sombras.

Resultado final:



El aporte real del proyecto

Aprendimos a **parsear geometría .obj**

Aplicamos matrices de **modelado**, **vista** y **proyección**

Diseñamos un sistema modular de carga y cámaras

Resolvimos problemas reales de **renderizado en Unity**

Experimentamos con el uso de **shaders** para lograr distintos efectos gráficos

Fin.